



LOAD DATA INTO DYNAMODB

By Nabil Ahmed | September 2026

Load Data into a DynamoDB Table

Table: ContentCatalog - Items returned (6)

Scan started on September 15, 2026, 17:09:40

☐	Id (Number) ▾	Authors ▾	ContentType ▾	Difficulty ▾
☐	3	[{"S": "Nex...	Project	Easy peasy
☐	2	[{"S": "Nex...	Project	Easy peasy
☐	203		Video	
☐	202		Video	
☐	201		Video	
☐	1	[{"S": "Nat...	Project	Easy peasy

Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a non-relational database service with AWS that organises data using key-value pairs.

How I used Amazon DynamoDB in this project

In today's project, I used Amazon DynamoDB to create five tables. I used CloudShell and CLI to load data into the tables quickly. I also compared DynamoDB to a relational database by understanding how items and attributes work.

One thing I didn't expect in this project was...

One thing I didn't expect in this project is that I could perform a lot of actions using AWS CLI instead of clicking around in the AWS Management Console. For example, creating tables, loading data, deleting tables.

This project took me...

This project took me one hour including documentation and understanding theory.

Create a DynamoDB table

DynamoDB tables organises data using items and attributes. Every single item is recorded with a set of attributes. Items can have any number of attributes (minimum 1, for the partition key value).

An attribute is a piece of data about an item in the DynamoDB table. For example, if an item was a specific student, attributes could be StudentName, ProjectsComplete, Email, SignupDate etc, the number of attributes for each item is flexible.

Table: NextWorkStudents - Items returned (1)

Scan started on September 15, 2026, 16:44:54

☐	StudentName (<i>String</i>) ▾	ProjectsComplete
☐	Nikko	4

Read and Write Capacity

Read capacity units (RCUs) and write capacity units (WCUs) are units that measure my DynamoDB table's performance. DynamoDB pricing is based on RCUs and WCUs so the more you use the more expensive the project.

Amazon DynamoDB's Free Tier covers 25 RCUs and WCUs monthly. I turned off auto scaling because this can result in higher charges if DynamoDB automatically scales up my operations (which charges more RCUs and WCUs)

Read/write capacity settings [Info](#)

Capacity mode
☐ On-demand
Simplify billing by paying for the actual reads and writes your application performs.

☒ Provisioned
Manage and optimize your costs by allocating read/write capacity in advance.

Read capacity

Auto scaling [Info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.
☐ On
☒ Off

Provisioned capacity units

Write capacity

Auto scaling [Info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.
☐ On
☒ Off

Provisioned capacity units

Using CLI and CloudShell

AWS CloudShell is an environment that lets us run code to interact with our AWS resources/services. Using AWS CloudShell is handy because AWS CLI is already installed, so we can run commands right away.

AWS CLI is a software that lets you interact with resources using commands instead of clicks in the console. This is a very practical software that is preferred over the AWS Management Console in day to day engineering operations in the real world.

I ran a CLI command in AWS CloudShell that created four new DynamoDB tables. The command is called "aws dynamodb create-table", and I could even define the table name and its attributes all within the command.

```
"CREATING"
~ $ aws dynamodb create-table \
>   --table-name Post \
>   --attribute-definitions \
>     AttributeName=ForumName,AttributeType=S \
>     AttributeName=Subject,AttributeType=S \
>   --key-schema \
>     AttributeName=ForumName,KeyType=HASH \
>     AttributeName=Subject,KeyType=RANGE \
>   --provisioned-throughput \
>     ReadCapacityUnits=1,WriteCapacityUnits=1 \
>   --query "TableDescription.TableStatus"
"CREATING"
~ $ aws dynamodb create-table \
>   --table-name Comment \
>   --attribute-definitions \
>     AttributeName=Id,AttributeType=S \
>     AttributeName=CommentDateTime,AttributeType=S \
>   --key-schema \
>     AttributeName=Id,KeyType=HASH \
>     AttributeName=CommentDateTime,KeyType=RANGE \
>   --provisioned-throughput \
>     ReadCapacityUnits=1,WriteCapacityUnits=1 \
>   --query "TableDescription.TableStatus"
```

Loading Data with CLI

I ran a CLI command in AWS CloudShell that loads multiple pieces of data into DynamoDB tables I set up in the previous step. This AWS CLI command is structured as "aws dynamodb batch-write-item --request-items file:///...".

```
nextworksampladata $ aws dynamodb batch-write-item --request-items file:///Forum.json
{
  "UnprocessedItems": {}
}
nextworksampladata $
nextworksampladata $ aws dynamodb batch-write-item --request-items file:///Post.json
{
  "UnprocessedItems": {}
}
nextworksampladata $
nextworksampladata $ aws dynamodb batch-write-item --request-items file:///Comment.json
{
  "UnprocessedItems": {}
}
```

Observing Item Attributes

Attributes				Add new attribute
☐ Attribute name	Value	Type	Remove	
☐ Id - Partition key	1	Number		
☒ Authors	Insert a field	List	Remove	
☐ ContentType	Project	String	Remove	
☐ Difficulty	Easy peasy	String	Remove	
☐ Price	0	Number	Remove	
☐ ProjectCategory	Storage	String	Remove	
☐ Published	☒ True ☐ False	Boolean	Remove	
☐ Title	Host a Website on Amazon S3	String	Remove	
☐ URL	aws-host-a-website-on-s3	String	Remove	

I checked a ContentCatalog item, which had the following attributes: Id, Authors, ContentType, Difficulty, Price, ProjectCategory, Published, Title and URL.

I checked another ContentCatalog item, which had a different set of attributes: Services, Title, VideoType etc.

Benefits of DynamoDB

A benefit of DynamoDB over relational databases is flexibility, because items can have their own set of attributes. This is great for scenarios where a table has different types of data, and some attributes don't apply to all data in that table.

Another benefit over relational databases is speed, because DynamoDB can quickly partition it's data to narrow down the search using Partition Keys. this is faster than relational databases, which need to scan entire tables to find specific pieces of data.

Table: ContentCatalog - Items returned (6)

Scan started on September 15, 2026, 17:09:40

☐	Id (Number)	Authors	ContentType	Difficulty
☐	3	[{"S": "Nex...	Project	Easy peasy
☐	2	[{"S": "Nex...	Project	Easy peasy
☐	203		Video	
☐	202		Video	
☐	201		Video	
☐	1	[{"S": "Nat...	Project	Easy peasy